

Building Agentic AI That Actually Works

Why Semantic Foundations Matter More Than the Models

Jeffrey Wallk

Principal Engineer, Semantic Governance & Knowledge Engineering

The Problem Nobody Wants to Talk About

The AI industry has moved fast—maybe too fast. We went from rule-based systems to generative models that can write poetry, and now we're racing toward autonomous agents that can supposedly run entire business processes. But here's what I've seen in deployments across telecommunications, aerospace, and pharmaceuticals: most agentic AI implementations fail quietly. They fail because organizations are building sophisticated autonomy on top of fundamentally messy, unstructured knowledge.

I've spent the last decade building semantic layers for Fortune 15 companies, managing over 30,000 data assets through knowledge graphs. The pattern is consistent: when you try to make AI agents autonomous without giving them proper semantic grounding, you get hallucinations at scale. You get agents that can't explain their decisions. You get automation that breaks in ways nobody predicted.

This isn't a technology problem. It's an architecture problem.

What Changes When AI Becomes Agentic

Traditional AI gave us automation for specific tasks. Generative AI gave us incredible content creation tools. Agentic AI is different—it's supposed to set goals, make plans, and execute complex workflows autonomously. The promise is real: imagine systems that can handle customer onboarding end-to-end, from HR paperwork to IT provisioning, without human handoffs. Or supply chain agents that reroute shipments in real-time when disruptions happen.

But autonomy amplifies every underlying weakness in your knowledge architecture. An agent that "hallucinates" doesn't just generate a wrong answer—it takes wrong *actions*. An agent that can't explain its reasoning doesn't just frustrate users—it creates compliance nightmares.

The solution isn't bigger models or more training data. It's fundamentally different infrastructure.

The Architecture That Makes Autonomy Trustworthy

Consider building a unified semantic layer. It's not a single ontology—that doesn't scale. Instead, think of it as two interrelated knowledge structures working together:

A Process-Centric Unifying Ontology

This captures the core business operations that cut across your entire organization. Order management. Fulfillment. Network operations. Customer lifecycle. These processes become first-class citizens in the ontology—not afterthoughts, not buried in documentation, but explicit entities with relationships, preconditions, and outcomes.

When you model Process, Event, Activity, Actor, and Goal as fundamental concepts, you create a common language. Different parts of the business can talk to each other. More importantly, different AI agents can coordinate without brittle point-to-point integrations.

Domain-Specific Ontologies

Here's where Domain-Driven Design becomes critical. Each bounded context—sales operations, warehouse logistics, compliance, customer support—needs its own rich, detailed ontology. These aren't generic data models. They're built with domain experts, capturing the nuances that only people who actually do the work understand.

In telecommunications, we built separate ontologies for network topology, service provisioning, and fault management. Each one is deep and specialized. But they all connect to the unifying process ontology, so an agent managing service delivery can reason across all three domains.

The magic happens at the intersection. A sales agent understands order states and customer relationships (from the sales ontology). A logistics agent understands warehouse capacity and shipping constraints (from the logistics ontology). The unifying ontology lets them collaborate on the complete order-to-delivery process without either agent needing to understand the other's entire domain.

How This Changes What AI Can Do

I've seen this architecture deliver outcomes that aren't possible with conventional approaches:

Grounding Small Language Models

We don't need massive foundational models for every task. A specialized SLM grounded in a rich domain ontology through RAG can outperform much larger models on domain-specific work. At one pharmaceutical client, we built an invoice validation system using a relatively small model. The model doesn't memorize every possible invoice format or business rule—it queries the Finance Operations knowledge graph in real-time, pulling current rules and past exceptions. When regulations change, we update the ontology, not the model.

Explainable Autonomy

An agentic system we deployed for fraud detection doesn't just flag suspicious transactions. When it escalates something to human review, it can trace its reasoning through the ontology: which transaction patterns it matched, which customer history it considered, which regulatory rules applied. The audit trail isn't reconstructed after the fact—it's inherent in how the system works.

Governed Collaboration

The most complex implementation I've led involved orchestrating seven specialized agents across customer onboarding. A master orchestrator uses the unifying process ontology to understand the complete workflow. It delegates identity verification to a compliance agent, account setup to a systems integration agent, and initial outreach to a communications agent. Each agent operates autonomously in its domain, but the orchestrator ensures nothing falls through the cracks and every handoff happens cleanly.

When the compliance agent detects an issue, it doesn't just stop—it uses the process ontology to trigger the correct exception handling workflow, notify the right people, and log everything for audit.

How Can We Build A Pragmatic Solution

I'm not going to pretend this is easy. Building semantic foundations takes time and expertise. But it's also not as overwhelming as it sounds if you approach it systematically.

Always Start With Process, Not Technology

Most organizations want to start by deploying agents. That's backwards. You need to understand your core business processes first. We typically begin with ISO standards and industry reference models to identify the critical value streams—the end-to-end processes that actually deliver value to customers.

For a retail company, that might be customer acquisition, order fulfillment, returns processing, and inventory replenishment. Map these at a high level. Identify the key stages, events, and handoffs. You're not building the ontology yet—you're just getting clear on what matters.

Build a the Unifying Ontology

Now you can model those core processes formally. Process becomes an entity with attributes: start time, end time, status, preconditions, postconditions. Events trigger processes or signal completion. Activities compose into processes. Actors perform actions.

This sounds abstract, but it's incredibly practical. When every system in your organization shares this common understanding of what a "process" is, integration becomes dramatically simpler. We use W3C standards (RDF, OWL) and ontology modeling tools for this work.

Focus On Critical Domains

Pick one high-value business area and build a proper domain ontology with the people who know that domain best. Not IT people—actual domain experts. For order management, that means understanding every state a sales order can be in, every business rule that governs transitions, every relationship to customers and products.

This ontology will be detailed and messy because the real world is detailed and messy. A SalesOrder might have relationships like hasCustomer, containsLineItem,

generatedByQuote, governedByContractTerms. Business rules like "cannot ship until payment confirmed and inventory allocated" become explicit, machine-readable logic.

Connect this domain ontology to your unifying process ontology. The SalesOrderLifecycle process relates to your generic Process entity, but it's enriched with all the domain-specific knowledge.

Make Knowledge Queryable

Now you can start populating vector databases with domain-specific content, enriched by your ontologies. Legal case documents go into a legal knowledge base with metadata derived from your legal ontology: case type, jurisdiction, relevant statutes, parties involved. Product documentation goes into a product knowledge base tagged with product categories, use cases, and troubleshooting patterns from your product ontology.

The embeddings themselves are more meaningful because they're contextualized by semantic metadata. When an agent queries for information, it's not just doing keyword matching—it's doing semantically-aware retrieval.

Build Specialized Intelligence

Only now do you start building the actual AI components. Develop small language models for specific tasks within bounded contexts. An invoice validation SLM. A customer query classification SLM. A network fault diagnosis SLM.

Integrate these models with RAG systems that query your domain-specific knowledge bases. The models stay small and fast because they're not trying to memorize everything—they're reasoning engines that pull facts from trusted sources.

Create Autonomous Agents

Each agent corresponds to a specific role or responsibility. A fraud detection agent. An inventory management agent. A customer service agent. Design them modularly with clear interfaces—perception modules that gather information, planning modules that decide what to do, execution modules that take action.

The key is that agents understand their local domain ontology. They know what entities exist, what relationships matter, what rules apply. They can reason within their domain with high fidelity.

Orchestrate Across Domains

This is where the unifying process ontology proves its value. Build an orchestrator (or use a multi-agent framework like CrewAI or AutoGen) that understands the global business process flow. It knows which agents have which capabilities, what the current process state is, and what needs to happen next.

For customer onboarding, the orchestrator sees that the process has seven stages. It delegates data capture to the CRM agent, identity verification to the compliance agent, and account activation to the systems integration agent. Each delegation is informed by the ontology—the orchestrator knows what information each agent needs and what outputs to expect.

When something goes wrong—and things will go wrong—the ontology provides the framework for intelligent error recovery.

Design Governance From the Start

Don't treat governance as an afterthought. The ontologies themselves are governance mechanisms. They define what data should look like, what rules must be followed, what decisions require human approval.

We typically build a compliance monitoring agent that continuously audits activity against policies defined in the ontology. When it detects a violation—say, a transaction that bypasses a required approval step—it doesn't just log an error. It can explain exactly which ontological rule was violated, trigger the correct remediation process, and escalate to the right people.

The explainability isn't bolted on. It's architectural.

How Do We Measure Value

Measure Outcomes

The business case for semantic foundations isn't theoretical. Organizations see returns in three measurable categories: process cycle time reduction, decision quality improvement, and change management cost avoidance. The key is designing for outcomes from the start.

Start With Process Bottlenecks

Pick one high-value process that's currently broken. Not the most complex process, not the most visible—the one where delays or errors create measurable business impact. Customer onboarding that loses deals. Exception handling that requires expensive escalations. Compliance checks that delay product launches.

Measure the current state precisely: How many handoffs? How long does each stage take? Where do exceptions occur? What percentage require human intervention? What's the error rate? These baseline metrics become your comparison points.

Design for Controlled Autonomy, Not Full Automation

Here's where most organizations go wrong: they try to automate everything immediately. That's dangerous with autonomous agents. Instead, design a graduated autonomy model:

Level 1: Agent recommends, human decides. The agent analyzes data, applies rules from the ontology, and presents options with explanations. Every decision requires human approval. This builds trust and captures edge cases you didn't anticipate.

Level 2: Agent decides within guardrails, human reviews. The agent handles routine cases autonomously but flags anything outside defined parameters. A fraud detection agent might auto-approve transactions under \$1,000 with clean profiles, but escalate anything suspicious or above threshold.

Level 3: Agent operates autonomously, human audits. The agent handles the full process end-to-end with comprehensive logging. Humans review patterns and outcomes, not individual decisions. This is where you get true scale, but only after proving reliability at Levels 1 and 2.

The ontology makes this progression possible. Decision rules, escalation thresholds, and approval requirements are explicit in the semantic layer. You're not changing code to adjust autonomy levels—you're updating policy definitions that every agent understands.

Track Decision Quality, Not Just Speed

Fast automation that makes bad decisions destroys value. The advantage of semantically-grounded agents is they can explain their reasoning, which means you can measure decision quality:

Precision: What percentage of agent decisions align with expert judgment when reviewed? In invoice validation, does the agent catch the same errors a senior accountant would catch?

Consistency: Do agents apply rules uniformly? A credit approval agent should handle similar applications identically. The ontology ensures this—rules are defined once, applied everywhere.

Explainability: Can the agent trace its reasoning through the ontology? When an insurance claim is denied, can it cite the specific policy terms, claim history, and underwriting rules that led to the decision? This isn't just for compliance—it's how you identify flawed logic before it scales.

Build in Economic Feedback Loops

Trustworthy engineering means measuring outcomes continuously and adjusting when reality diverges from expectations. Connect agent decisions to business results:

A customer service agent routes inquiries to appropriate teams. Track resolution time and customer satisfaction by routing decision. If the agent consistently misroutes technical questions, that's measurable. Update the customer issue ontology with better classification criteria.

A supply chain agent makes inventory allocation decisions. Measure stockout rates and carrying costs against the agent's predictions. When actual demand deviates from agent forecasts, the ontology captures the new patterns—seasonality factors, regional variations, promotional impacts—making future decisions more accurate.

This feedback loop is architectural. Agents don't just execute—they generate data about their own decision quality. That data flows back into ontology refinement, creating a system that gets smarter through use.

Quantify the Change Management Dividend

Traditional automation creates technical debt. Every business rule change requires code updates, testing, deployment cycles. A simple policy change—"require director approval

for expenses over \$5,000 instead of \$10,000"—might touch a dozen systems, take weeks to implement, and risk breaking something.

With semantic foundations, that same change happens in the ontology. You update the approval threshold rule. Every agent that evaluates expense approvals immediately applies the new threshold. No code changes. No deployment. No testing cycle. The policy is data, not logic buried in software.

This isn't a minor convenience—it's a fundamental shift in organizational agility. When you can change business rules at the speed of business decisions, you eliminate a massive class of technical debt. Track how many "simple policy changes" currently require IT work. That's your change management baseline. After implementing semantic architecture, those changes happen in hours, not sprints.

Real Numbers From Real Deployments

What does this look like in practice? Here are outcome patterns from actual implementations:

Exception handling processes that required 12-15 handoffs and 5-7 business days now complete in under 4 hours with 2-3 touch points. The agents handle investigation, data gathering, and routing. Humans make final judgment calls. Cycle time reduction: 85%. Error rate: down 60%.

Compliance validation that consumed 40 FTE hours per week now operates continuously with agent monitoring and human audit review. The compliance agent checks every transaction against regulatory ontology rules in real-time. Policy updates that used to take 6-8 weeks to propagate now deploy instantly. Labor redeployment: 35 FTEs to higher-value work. Violation detection: up 40%.

Complex routing decisions—which facility handles which order based on inventory, capacity, geography, and contract terms—that required expert judgment now happen automatically with 94% accuracy. The agent explains its reasoning: "Routed to Atlanta facility because: (1) item in stock, (2) falls within Southeast territory per customer contract, (3) capacity available for 2-day delivery commitment." When experts disagree with routing, their corrections update the ontology, improving future decisions.

The cost savings are substantial—smaller models, optimized processes, fewer errors. But the real return is strategic flexibility. Organizations can evolve business rules, launch new services, and respond to market changes without being constrained by automation that's too rigid to adapt or too opaque to trust.

Why Most Organizations Won't Do This

Building proper semantic foundations requires patience and expertise that most organizations don't have or won't invest in. It requires getting domain experts and data engineers in the same room for weeks. It requires resisting the pressure to deploy something quickly and instead doing the foundational work.

It also requires accepting that knowledge engineering is a discipline, not a weekend project. You need people who understand ontology design, who can facilitate knowledge elicitation with domain experts, who can architect knowledge graphs for performance and maintainability. Progressive investments can be used to fund ongoing architectural rationalization and eliminating redundant systems. The returns from agentic AI built on these foundations are different in kind, not just degree:

- Fraud detection that used to require teams of analysts now runs continuously with a handful of agents, catching patterns that humans miss while explaining every decision.
- Onboarding processes that took weeks and multiple handoffs now complete in hours, with higher accuracy and full audit trails.
- Supply chain disruptions that would have cascaded into service failures now trigger automatic rerouting, with agents coordinating across logistics, inventory, and customer communication.

The cost savings are substantial—smaller models, less human intervention, fewer errors. But the real value is agility. When you need to change a business rule or add a new process step, you update the ontology. Every agent that relies on that knowledge immediately understands the change. No retraining, no redeployment, no months-long change management.

The organizations that get this right—and I've worked with a handful—treat semantic infrastructure the way they treat security or data governance. As foundational, ongoing, strategic capability, not a project with an end date.

What Comes Next

We're at an inflection point. The next generation of AI won't be about smarter models in isolation. It will be about ecosystems of specialized agents working together, grounded in rich semantic understanding of business domains and processes.

The technical pieces are ready. Knowledge graphs are mature. Vector databases are sophisticated. Small language models are surprisingly capable. Multi-agent frameworks are emerging. The question is whether organizations will do the hard architectural work to make autonomy trustworthy.

I've spent my career building the foundations that make complex systems reliable at scale. The pattern is always the same: shortcuts early create compounding problems later. Do the semantic work now, or spend the next decade managing the chaos of autonomous systems built on sand.

The timing is right for applying engineering discipline to realize the benefits of AI and position organizations for serious deployments of agentic AI that actually work in production, that meets regulatory requirements, that you can explain to auditors and trust with critical operations.

Let's Build AI Right.

About the Author

Jeffrey Wallk is Principal Engineer in Semantic Governance and Knowledge Engineering at The Value Enablement Group. He spent the last 5 years at Verizon where he has established Knowledge Engineering as a formal discipline at the Fortune 15 company. He architects unified semantic knowledge layers and has pioneered RAG 2.0 implementations integrating knowledge graphs with GenAI. He served as Co-Lead of the Meta Model Team for the Business Architecture Guild for 3 years, played active roles with the Enterprise Knowledge Graph Foundation and Digital Twin Consortium. His current focus is to work with the Knowledge Systems WG at INCOSE to develop a participatory model across WGs to build a bridge for human and machine understanding as AI expands into every area of engineering.



ANTICIPATE | INNOVATE | ACCELERATE